

# Azure Setup Instructions for ChordSheets

Everything below is done in the **Azure Portal** (<https://portal.azure.com>). No commands needed.

## Step 1: Create a Resource Group

A resource group is just a folder that holds all your Azure resources together.

1. Go to **Azure Portal** → search for **"Resource groups"** in the top search bar
2. Click **"+ Create"**
3. Fill in:
  - **Subscription:** Your subscription (likely "Azure subscription 1" or similar)
  - **Resource group name:** chordsheets-rg
  - **Region:** East US (or whichever is closest to you)
4. Click **"Review + create"** → **"Create"**

## Step 2: Create Azure Cosmos DB (Free Tier)

This is the database that stores song metadata (title, artist, tags, etc.).

1. Search for **"Azure Cosmos DB"** in the top search bar
2. Click **"+ Create"**
3. Select **"Azure Cosmos DB for NoSQL"** → click **"Create"**
4. Fill in:
  - **Subscription:** Same as before
  - **Resource group:** chordsheets-rg
  - **Account name:** chordsheets-db (must be globally unique — if taken, try chordsheets-db-123 or similar)
  - **Location:** Same region as your resource group (e.g., East US )
  - **Capacity mode:** Provisioned throughput
  - ☒ **Check "Apply Free Tier Discount"** (this gives you 1000 RU/s and 25 GB free)
5. Click **"Review + create"** → **"Create"**
6. Wait for deployment to complete (takes 2-5 minutes)

## 2b: Create the Database and Container

1. Once deployed, click **"Go to resource"**
2. In the left sidebar, click **"Data Explorer"**
3. Click **"New Container"** (at the top)
4. Fill in:
  - **Database id:** Select **"Create new"** → type `chordsheets`
  - ☒ **Check "Share throughput across containers"**
  - **Database throughput:** Select **"Manual"** → set to `400 RU/s` (this is within free tier)
  - **Container id:** `songs`
  - **Partition key:** `/id`
5. Click **"OK"**

## 2c: Get Your Cosmos DB Connection Info

1. In the left sidebar, click **"Keys"** (under "Settings")
2. You will see:
  - **URI** — copy this (looks like `https://chordsheets-db.documents.azure.com:443/` )
  - **PRIMARY KEY** — copy this (long base64 string)
3. **Save both of these** — you'll need them in Step 5

## Step 3: Create Azure Storage Account (Blob Storage)

This is where your PDF files are stored.

1. Search for **"Storage accounts"** in the top search bar
2. Click **" + Create "**
3. Fill in:
  - **Subscription:** Same as before
  - **Resource group:** `chordsheets-rg`
  - **Storage account name:** `chordsheetsblob` (must be globally unique, lowercase, no hyphens — if taken try `chordsheetsblob123` )
  - **Region:** Same as before (e.g., `East US` )
  - **Performance:** **Standard**
  - **Redundancy:** **LRS** (Locally-redundant storage — cheapest option, fine for personal use)
4. Click **"Review + create"** → **"Create"**
5. Wait for deployment, then click **"Go to resource"**

## 3b: Create the PDF Container

1. In the left sidebar, click "**Containers**" (under "Data storage")
2. Click "**+ Container**"
3. Fill in:
  - **Name:** pdfs
  - **Public access level:** **Private** (no anonymous access)
4. Click "**Create**"

## 3c: Enable CORS on Blob Storage

This allows the browser to fetch PDFs directly.

1. In the left sidebar, click "**Resource sharing (CORS)**" (under "Settings")
2. Under "**Blob service**", add a row:
  - **Allowed origins:** \*
  - **Allowed methods:** GET
  - **Allowed headers:** \*
  - **Exposed headers:** \*
  - **Max age:** 3600
3. Click "**Save**"

## 3d: Get Your Storage Connection String

1. In the left sidebar, click "**Access keys**" (under "Security + networking")
2. Click "**Show**" next to the first key's Connection string
3. Copy the **Connection string** (starts with `DefaultEndpointsProtocol=https;AccountName=...`)
4. **Save this** — you'll need it in Step 5

# Step 4: Create Azure Static Web App

This hosts your website and API.

## 4a: Push Code to GitHub First

Before creating the Static Web App, your code needs to be in a GitHub repository:

1. Go to **GitHub** (<https://github.com>) → create a new repository called chordsheets

2. On your computer, open a terminal in your project folder and run:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/chordsheets.git
git push -u origin main
```

## 4b: Create the Static Web App

1. Back in **Azure Portal**, search for **"Static Web Apps"**

2. Click **"+ Create"**

3. Fill in:

- **Subscription:** Same as before
- **Resource group:** chordsheets-rg
- **Name:** chordsheets-app
- **Plan type:** Free
- **Region:** Pick the closest to you
- **Source:** GitHub

4. Click **"Sign in with GitHub"** and authorize Azure

5. After signing in, fill in:

- **Organization:** Your GitHub username
- **Repository:** chordsheets
- **Branch:** main

6. Under **Build Details:**

- **Build Presets:** Custom
- **App location:** /
- **Api location:** api
- **Output location:** dist

7. Click **"Review + create"** → **"Create"**

Azure will automatically create a GitHub Actions workflow and trigger your first deployment. The first deploy may fail because we haven't set the environment variables yet — that's okay, we'll fix that next.

## Step 5: Configure Environment Variables

This connects your Static Web App to Cosmos DB and Blob Storage.

1. Go to your Static Web App resource ( `chordsheets-app` )
2. In the left sidebar, click **"Configuration"** (under "Settings")
3. Click **" + Add "** for each of the following application settings:

| Name                   | Value                                                                                                    |
|------------------------|----------------------------------------------------------------------------------------------------------|
| COSMOS_ENDPOINT        | The URI you copied from Step 2c (e.g.,<br><code>https://chordsheets-db.documents.azure.com:443/</code> ) |
| COSMOS_KEY             | The PRIMARY KEY you copied from Step 2c                                                                  |
| COSMOS_DATABASE        | <code>chordsheets</code>                                                                                 |
| COSMOS_CONTAINER       | <code>songs</code>                                                                                       |
| BLOB_CONNECTION_STRING | The Connection string you copied from Step 3d                                                            |
| BLOB_CONTAINER         | <code>pdfs</code>                                                                                        |

4. After adding all 6, click **"Save"** at the top

## Step 6: Trigger a New Deployment

The first deployment likely failed because the environment variables weren't set. Now that they are:

1. Go to **GitHub** → your `chordsheets` repository
2. Click the **"Actions"** tab
3. You should see the failed workflow run
4. Click on it → click **"Re-run all jobs"**
5. Wait for it to complete (2-3 minutes)

Alternatively, just make a small commit and push — it will auto-deploy.

## Step 7: Access Your App

1. Go back to your Static Web App in Azure Portal
2. On the **Overview** page, you'll see a **URL** (looks like `https://xxxx-xxxx.azurestaticapps.net` )
3. Click it — your ChordSheets app should load!
4. **On your phone:** Open that URL in Safari/Chrome → tap "Add to Home Screen" → it becomes a PWA app icon
5. **On your iPad:** Same URL — use the "Add" tab to upload songs

## Summary of What You Created

| Resource        | Name            | Purpose                  | Cost             |
|-----------------|-----------------|--------------------------|------------------|
| Resource Group  | chordsheets-rg  | Container for everything | Free             |
| Cosmos DB       | chordsheets-db  | Song metadata database   | Free (free tier) |
| Storage Account | chordsheetsblob | PDF file storage         | ~\$0.10/month    |
| Static Web App  | chordsheets-app | Website + API hosting    | Free             |

**Total monthly cost: ~\$0.10**

## Troubleshooting

### "The app loads but shows errors when I try to add a song"

- Check that all 6 environment variables are set correctly in the Static Web App Configuration
- Make sure the Cosmos DB database and container names match ( `chordsheets` and `songs` )
- Make sure the Blob container name matches ( `pdfs` )

### "The deployment failed"

- Go to GitHub → Actions tab → click the failed run → read the error logs
- Common issue: the GitHub Actions workflow file was modified by Azure when creating the Static Web App. If there are two workflow files, delete the Azure-generated one and keep the one in

`.github/workflows/deploy.yml`

## "I can't find my Static Web App URL"

- Azure Portal → Static Web Apps → click your app → the URL is on the Overview page

## "PDFs don't load"

- Make sure CORS is enabled on the Storage Account (Step 3c)
- Make sure the `BLOB_CONNECTION_STRING` is correct in your Static Web App configuration